Interview Guide

Software Team Manager: Camlin (Beograd)

Detaljan vodič sa pitanjima koje bi iskusan tim lider postavio na intervjuu, uz model odgovore. Pokriva liderstvo, upravljanje talentima, Agile, inženjersku kulturu, .NET tehnička pitanja, SaaS/IoT arhitekturu, bihejvioralna pitanja i kratke odgovore na engleskom.

Pozicija	Software Team Manager, Software Engineering, Beograd
Domen	Web SaaS aplikacije za prikupljanje, analizu i vizualizaciju podataka sa IoT uređaja koji prate elektro-mrežu
Ključno	Min. 4 godine vođenja inženjerskih timova, people management, web/cloud-native tehnologije, SaaS, razvojna pozadina, tečan engleski
Poželjno	Python/.NET/TypeScript/Bash/PowerShell, monitoring (Prometheus, Datadog, New Relic), mreže, bezbednost, identity management
Vrednosti	Stalno poboljšanje, iskrena povratna informacija, saradnja, radoznalost i slušanje, odgovornost (ljudi, profit, planeta)

Sadržaj

1. Liderstvo i upravljanje ljudima (Line Management)
2. Upravljanje talentima: zapošljavanje, zadržavanje, obuka
3. Agile, isporuka i saradnja između funkcija
4. Inženjerska kultura, kvalitet i tehnička strategija
5. Tehnička pitanja: .NET i web backend
6. SaaS, cloud-native i IoT domen (praćenje elektro-mreže)
7. Bihejvioralna pitanja (koristite STAR metodu)
8. Kratki odgovori na engleskom (poziciji treba tečan engleski)
9. Pitanja koja vi postavljate kompaniji

Kako koristiti vodič

Odgovori su napisani u prvom licu kao model odgovori. Nemojte ih učiti napamet: zamenite primere svojim stvarnim iskustvom, brojkama i imenima projekata. Linija u kurzivu ispod svakog pitanja objašnjava šta intervjuer zapravo procenjuje.

Za bihejvioralna pitanja pripremite 5-6 sopstvenih priča (STAR) koje možete da prilagodite različitim pitanjima.

1. Liderstvo i upravljanje ljudima (Line Management)

P1. Kako biste opisali svoj stil vođenja?

Šta se procenjuje: Samosvest, prilagodljivost, stil koji odgovara kulturi (kolaboracija, odgovornost).

Opisao bih ga kao „servant leadership“ sa jasnim očekivanjima. Moj posao je da uklonim prepreke, dam timu kontekst (zašto radimo to što radimo) i autonomiju u tome kako to rade. Stil prilagođavam osobi: juniorima dajem više strukture i čestu povratnu informaciju, a seniorima više slobode i prostora za vlasništvo nad odlukama.

Primer: kod novog juniora sam prvih mesec dana radio kratke dnevne check-in-e i pair programming, a kasnije prešao na nedeljne 1:1 sastanke i delegiranje celih funkcionalnosti.

P2. Kako planirate resurse u timu sa različitim skillsetom (frontend, backend, data, DevOps)?

Šta se procenjuje: Planiranje kapaciteta, rizik od zavisnosti od pojedinaca, realnost.

Krećem od prioriteta proizvoda sa Product Ownerom i Project Managementom, pa mapiram potrebne veštine na zadatke. Pravim jednostavnu matricu veština i identifikujem „bus factor“ tamo gde samo jedna osoba zna neki deo sistema.

- Kapacitet računam realno: odmori, bolovanja, održavanje, bagovi, sastanci. Obično rezervišem 15-20% kapaciteta.
- Planiram na osnovu stvarne velocity, ne na osnovu želja.
- Aktivno radim cross-training (pair programming, rotacija na code review-ima) da bi tim bio otporniji.
- Kad vidim manjak veštine, biram između obuke, zapošljavanja i spoljne pomoći, u zavisnosti od toga da li je potreba trajna ili privremena.

P3. Kako biste postupili sa inženjerom koji ne ispunjava očekivanja?

Šta se procenjuje: Empatija + odgovornost, strukturiran proces, poštovanje prema osobi.

Prvo pokušam da razumem uzrok: da li je u pitanju veština, nejasna očekivanja, motivacija, loš fit u timu ili privatne okolnosti. Ne pretpostavljam.

- Privatan razgovor, konkretni primeri (ne opšte fraze) i pitanje: „Šta ti je potrebno od mene?“
- Zajedno dogovorimo merljive ciljeve i rokove (npr. plan 30/60/90 dana) i redovne provere.
- Dokumentujem razgovore i uključujem HR ako nema napretka.

Cilj je da osoba uspe. Ako ipak ne uspe, odluka je poštena i transparentna, jer je osoba tokom celog procesa znala gde stoji.

P4. Kako izgledaju vaši 1:1 sastanci?

Šta se procenjuje: Da li su 1:1 fokusirani na osobu ili samo na status zadataka.

Održavam ih redovno (nedeljno ili na dve nedelje, 30 min) i oni su prostor osobe, a ne moj. Agenda je zajednička.

- Kako se osećaš? Šta te blokira? Šta te iznervira?
- Povratna informacija u oba smera (tražim i feedback o sebi).
- Karijerni razvoj: gde želiš da budeš za godinu-dve?
- Beleške i dogovorene akcije, i obavezno pratim da sam ispunio svoj deo.

Statusi zadataka idu na stand-up i board, ne na 1:1.

P5. Kako dajete povratnu informaciju, naročito negativnu?

Šta se procenjuje: Konkretnost, pravovremenost, kultura otvorene i iskrene povratne informacije (vrednost kompanije).

Pravovremeno i privatno, a pohvale javno. Koristim model situacija-ponašanje-uticaj (SBI): opišem konkretnu situaciju, šta je osoba uradila i kakav je to uticaj imalo, bez etiketiranja ličnosti.

Primer: umesto „loše pišeš kod“, kažem „u PR-u od utorka nedostaju testovi za novu logiku, a zbog toga smo imali bag u produkciji“. Zatim pitam kako vidi situaciju i zajedno tražimo rešenje. Takođe tražim feedback od tima o sebi, jer kultura iskrene povratne informacije mora da počne od lidera.

P6. Kako motivišete tim kada je pod pritiskom ili kada je posao monoton?

Šta se procenjuje: Razumevanje motivacije, zaštita tima, smisao posla.

Povezujem posao sa smislom: u ovom domenu naš softver pomaže da se prate i spreče kvarovi na elektro-mreži, što je stvaran uticaj. Pod pritiskom štitim tim od nepotrebnog šuma, jasno prioritizujem i govorim „ne“ kada je potrebno.

Kod monotonije dajem autonomiju, priliku da rade nešto novo (tech spike, hackathon, dani za učenje) i slavim isporuke. Motivaciju ne pretpostavljam, nego pitam svakog šta mu je važno: neko traži izazov, neko stabilnost, neko mentorisanje.

P7. Kako rešavate konflikt između dva inženjera?

Šta se procenjuje: Neutralnost, fokus na činjenice, brzo reagovanje.

Ne ignorišem ga, jer se konflikti ne rešavaju sami. Prvo razgovaram sa svakim pojedinačno da razumem perspektive, a zatim ih okupim oko zajedničkog cilja i činjenica, ne ličnosti.

- Kod tehničkih sporova tražim podatke: benchmark, proof of concept, ADR dokument sa prednostima i manama.
- Ako se ne dogovore, ja donosim odluku, objasnim razloge i tražim „disagree and commit“.
- Nakon toga pratim da odnosi budu zdravi.

P8. Kako vodite performance review i kalibraciju ocena?

Šta se procenjuje: Fer procena, objektivni kriterijumi, kontinuiran proces.

Review ne sme da bude iznenađenje: tokom cele godine imam kontinuirani feedback i beležim konkretna dostignuća. Koristim jasnu matricu kompetencija po nivoima (junior, medior, senior, lead) i ocenjujem učinak (šta je postignuto), ali i ponašanje (kako je postignuto, u skladu sa vrednostima).

Na kalibraciji sa drugim menadžerima proveravam doslednost i izbegavam pristrasnosti (recency bias, halo efekat). Svaki review završava planom razvoja i konkretnim ciljevima.

P9. Kako upravljate timom koji radi hibridno ili na različitim lokacijama?

Šta se procenjuje: Komunikacija, inkluzivnost, dokumentacija.

Ključno je pisanje: odluke, dogovori i arhitektura moraju biti dokumentovani da niko ne zavisi od „hodničkih razgovora“. Definišem sate preklapanja za sinhronu komunikaciju, a ostalo radimo asinhrono.

Pazim na inkluzivnost (da udaljeni članovi ne budu zapostavljeni na sastancima), a lični kontakt gradim redovnim timskim okupljanjima. Kod hibrida dogovaramo dane u kancelariji za radionice i planiranje.

2. Upravljanje talentima: zapošljavanje, zadržavanje, obuka

P10. Opišite vaš proces zapošljavanja inženjera od početka do kraja.

Šta se procenjuje: Strukturiran proces, objektivni kriterijumi, iskustvo kandidata.

- Definisanje uloge: zajedno sa timom odredim šta zaista nedostaje (veštine, senioritet) i pišem realan opis.
- Sourcing: LinkedIn, preporuke zaposlenih, zajednice, saradnja sa HR-om.
- Screening: kratak razgovor (motivacija, komunikacija, engleski).
- Tehnički krug: praktičan zadatak ili pair programming na realnom problemu i diskusija o dizajnu sistema (ne testovi sa pamćenjem algoritama).
- Krug sa timom i vrednostima: saradnja, davanje i primanje povratne informacije.
- Odluka: svi intervjueri ocenjuju prema istim kriterijumima pre diskusije, da se izbegne uticaj jačeg glasa.
- Candidate experience: brza i iskrena povratna informacija i pozitivni i negativni kandidati.

P11. Kako procenjujete tehničku sposobnost kandidata kada ste sada više menadžer nego programer?

Šta se procenjuje: Autentičnost, uključivanje tima.

Ostajem tehnički dovoljno aktuelan da postavljam prava pitanja (arhitektura, kompromisi, debugging pristup), a dubinsku proveru prepuštam senior inženjerima u timu. Fokusiram se na to kako kandidat razmišlja, ne samo na to šta zna: kako formuliše probleme, koje kompromise vidi, kako reaguje kada ne zna odgovor.

P12. Kandidat je tehnički odličan, ali pokazuje znakove lošeg timskog rada. Šta radite?

Šta se procenjuje: Prioritet vrednosti i kulture, ne samo veštine.

Ne zapošljavam ga. Jedan „brilliant jerk“ može da uništi dinamiku tima i dovede do odlaska dobrih ljudi. Pre odluke proveravam da li su moji utisci zasnovani na konkretnim primerima iz intervjua (kako govori o bivšim kolegama, kako reaguje na kritiku) i tražim mišljenje drugih intervjuera. Vrednosti kompanije (saradnja, otvorenost, odgovornost) su kriterijum, ne bonus.

P13. Kako zadržavate top talente?

Šta se procenjuje: Proaktivnost, razumevanje motivacije, konkretne mere.

Zadržavanje počinje mnogo pre otkaza.

- Redovni 1:1 i „stay interview“ (šta bi te ovde zadržalo, šta bi te odvuklo?).
- Jasan razvojni put (tehnička i menadžerska karijera), mentorstvo.
- Zanimljivi problemi, autonomija i vlasništvo.
- Fer i transparentna naknada, a gde ne mogu da utičem, to otvoreno komuniciram i eskaliram.
- Praćenje signala upozorenja: pad angažovanja, povlačenje sa sastanaka, manje inicijative.

Ako neko ipak ode, radim exit razgovor i učim iz njega.

P14. Kako identifikujete i planirate obuke za inženjere?

Šta se procenjuje: Povezanost obuke sa potrebama tima i proizvoda.

Spajam tri izvora: lične razvojne ciljeve (iz 1:1), potrebe tima (npr. nedostatak znanja iz observability-ja ili bezbednosti) i strategiju proizvoda (npr. prelazak na novu tehnologiju). Zatim biram format: interne radionice, brown bag sesije, sertifikati (Azure/AWS), konferencije, knjige i vreme za učenje (npr. nekoliko sati nedeljno). Merim rezultat: da li je znanje primenjeno u radu, a ne samo da li je kurs završen.

P15. Kako izgleda onboarding novog člana tima?

Šta se procenjuje: Struktura i brzina uključivanja.

Pripremim plan 30/60/90 dana pre nego što osoba dođe: oprema i pristupi spremni prvog dana, buddy iz tima, jasna dokumentacija (arhitektura, lokalno okruženje, procesi) i prvi mali zadatak koji se stvarno isporučuje u prvoj nedelji. Redovno proveravam kako se osoba oseća i tražim feedback o samom onboardingu da ga poboljšam.

P16. Kako gradite balans između seniora i juniora u timu?

Šta se procenjuje: Dugoročno razmišljanje.

Tim sa samo seniorima je skup i teško se održava, a tim sa samo juniorima je spor i rizičan. Težim zdravom odnosu gde seniori mentorišu, a juniori donose energiju i sveže poglede. Mentorstvo uključujem u očekivanja seniora i priznajem ga u njihovom napredovanju, da ne bude „posao posle posla“.

3. Agile, isporuka i saradnja između funkcija

P17. Kako vodite Agile razvoj u praksi? Scrum, Kanban ili nešto treće?

Šta se procenjuje: Pragmatičnost, a ne dogma.

Biram ono što odgovara timu i prirodi posla. Scrum je dobar za planirani rad na funkcionalnostima, a Kanban za održavanje i neplanirani rad. Često koristim hibrid („Scrumban“). Ključne elemente čuvam bez obzira na naziv: kratki ciklusi povratne informacije, transparentan backlog, retrospektive koje stvarno proizvode akcije i vidljiv tok rada. Ceremonije ne radim zbog ceremonija; ako neki sastanak ne daje vrednost, ukidam ga ili menjam.

P18. Kako merite uspešnost tima?

Šta se procenjuje: Metrike koje ne guraju pogrešno ponašanje.

Koristim DORA metrike: učestalost isporuke, lead time za promene, stopu neuspešnih promena i vreme oporavka (MTTR), uz cycle time i kvalitet (bagovi u produkciji). Izbegavam metrike aktivnosti kao što su broj linija koda ili broj commit-ova, jer podstiču loše ponašanje. Metrike koristim za učenje i razgovor u timu, a ne za poređenje pojedinaca. Dodajem i kvalitativne signale: zadovoljstvo tima i zadovoljstvo korisnika.

P19. Kako sarađujete sa Product Ownerima i Project Managementom?

Šta se procenjuje: Partnerski odnos, jasna podela odgovornosti.

PO odgovara za šta i zašto, tim za kako i koliko dugo. Redovno radimo zajednički refinement, gde inženjeri rano uključeni postavljaju pitanja i daju procene izvodljivosti. Sa PM-om delim transparentan status, rizike i zavisnosti i nikada ne obećavam rok bez procene tima. Ako PO traži previše, ne kažem prosto „ne“, nego prikažem kompromise: obim, rok, kvalitet i resursi, pa zajedno biramo.

P20. Rok se približava, a tim kasni. Šta radite?

Šta se procenjuje: Rešavanje pod pritiskom, transparentnost.

- Prvo utvrdim realno stanje: šta je zaista ostalo i koji su rizici.
- Rano i otvoreno obavestim PM/PO i Directora, jer loše vesti ne postaju bolje sa vremenom.
- Predložim opcije: smanjenje obima (MVP), pomeranje roka, dodatni resursi (uz svest o Brooks-ovom zakonu).
- Ne žrtvujem kvalitet i bezbednost, a ne pritiskam tim na prekovremeni rad kao standard.
- Nakon isporuke radim retrospektivu: zašto smo kasnili i kako da procene budu bolje.

P21. Kako postupate kada su zahtevi nejasni ili se stalno menjaju?

Šta se procenjuje: Rad u neizvesnosti.

Skraćujem petlju povratne informacije: manji inkrementi, rani prototipi i demo-i za stakeholdere. Zajedno sa PO-om postavljam „definition of ready“ da zahtev ne uđe u sprint bez prihvatnih kriterijuma. Promene su normalne u Agile-u, ali ih činim vidljivim: svaka promena u toku sprinta ima cenu i o njoj se otvoreno razgovara.

P22. Kako procenjujete i planirate rad? Šta mislite o story points i procenama?

Šta se procenjuje: Realnost, zrelost.

Procene su razgovor o nesigurnosti, a ne obećanje. Koristim relativne procene (story points) ili još radije mali, približno jednaki zadaci i praćenje throughput-a i cycle time-a za prognoze. Dugoročne rokove dajem kao opseg sa verovatnoćom (npr. na osnovu istorijskih podataka), ne kao jedan datum.

P23. Kako olakšavate donošenje odluka između više timova i funkcija?

Šta se procenjuje: Facilitacija, jasno vlasništvo.

Pre svega razjasnim ko donosi odluku (npr. RACI ili DACI okvir) i koji su kriterijumi. Pripremam kratak pisani dokument sa opcijama, pa diskusija na sastanku bude fokusirana. Ključne odluke beležim (ADR) i delim, da se ne vraćamo na iste rasprave. Rano uključujem sve pogođene strane, jer je „iznenađenje na kraju“ najskuplje.

4. Inženjerska kultura, kvalitet i tehnička strategija

P24. Kako biste uveli kulturu visokih performansi u tim koji to trenutno nema?

Šta se procenjuje: Postepenost, uključivanje tima, merljivost (ključni zahtev pozicije).

Ne radim revoluciju. Prvo se naslušam: razgovaram sa svakim, analiziram bolne tačke (spori deploy-evi, bagovi, nejasnoće). Zatim biram 2-3 najvažnija poboljšanja i uvodim ih zajedno sa timom.

- Brze pobede: automatizovani build i testovi u CI, obavezan code review.
- Srednji rok: CI/CD sa automatskim deploy-em, definicija „gotovo“, observability.
- Kulturološki: blameless postmortem-i, deljenje znanja, priznavanje dobrih praksi.

Merim DORA metrike pre i posle da pokažem efekat, i redovno proveravam sa timom da li praksa pomaže ili je teret.

P25. Kako balansirate tehnički dug i isporuku novih funkcionalnosti?

Šta se procenjuje: Poslovni jezik, prioritizacija.

Tehnički dug činim vidljivim i prevodim ga u poslovni uticaj: usporavanje isporuke, rizik od incidenata, cena održavanja. Sa PO-om dogovaram stalni udeo kapaciteta (npr. 15-20% po sprintu) i prioritizujem dug koji najviše košta. Nije svaki dug loš: ponekad svesno uzimamo dug radi brzine, ali ga evidentiramo i planiramo otplatu.

P26. Kako pristupate standardizaciji tehnologija, a da ne ugušite inovaciju?

Šta se procenjuje: Balans (direktno iz opisa posla).

Standardizujem ono što smanjuje trošak i rizik (logging, autentikacija, CI/CD šabloni, struktura projekata, osnovni stack), a inovaciji ostavljam kontrolisan prostor: tech spike-ovi, proof of concept i „tech radar“ sa kategorijama (adopt, trial, assess, hold). Nova tehnologija se uvodi kada rešava stvarni problem, uz procenu cene održavanja, ne zbog hype-a.

P27. Kako izgleda dobar code review proces?

Šta se procenjuje: Kultura učenja, ne kontrole.

Mali PR-ovi, brza povratna informacija (dogovoreni SLA, npr. isti dan), automatizovane provere (linter, testovi, statička analiza) da ljudi ne gube vreme na formatiranje, a ljudski review se fokusira na dizajn, čitljivost, bezbednost i testove. Ton je saradnički: pitanja umesto naređenja, i obavezno primećivanje dobrih rešenja. Review je i način deljenja znanja, ne samo kapija za kvalitet.

P28. Kako pristupate incidentima u produkciji i postmortem-ima?

Šta se procenjuje: Zrelost i blameless kultura.

Tokom incidenta: jasne uloge (incident commander, komunikacija, tehnički lider), prvo ublažavanje (rollback, feature flag), pa tek onda analiza uzroka. Nakon toga blameless postmortem: vremenska linija, uzroci (sistemski, ne lični), šta je dobro prošlo i konkretne akcije sa vlasnikom i rokom. Pratim da se akcije stvarno izvrše, jer je postmortem bez akcija samo beleška.

P29. Kakvu strategiju testiranja biste preporučili?

Šta se procenjuje: Piramida testova, praktičnost.

Piramida: mnogo brzih unit testova za poslovnu logiku, solidan sloj integracionih testova (baza, API, poruke) i manji broj end-to-end testova za kritične tokove. Uz to contract testovi među servisima, testovi performansi za kritične putanje i monitoring u produkciji kao „poslednja linija“ testiranja. Cilj nije procenat pokrivenosti, nego poverenje u promene.

P30. Kako dokumentujete arhitekturu i odluke?

Šta se procenjuje: Održivost znanja.

Lagana dokumentacija blizu koda: README, dijagrami (C4 model), ADR za značajne odluke i runbook-ovi za operativne procedure. Dokumentacija ide kroz isti review proces kao kod, da ostaje aktuelna. Bolje je malo dokumentacije koja je tačna nego mnogo zastarele.

5. Tehnička pitanja: .NET i web backend

P31. Koja je razlika između .NET Framework-a i savremenog .NET-a (ranije .NET Core)? Koju verziju biste birali za nov projekat?

Šta se procenjuje: Poznavanje ekosistema i politike podrške.

Savremeni .NET je open-source, višeplatformski (Windows, Linux, macOS), brži, modularniji i dobro radi u kontejnerima i cloud-u. .NET Framework je samo za Windows i u režimu održavanja. Za nov projekat biram aktuelnu LTS verziju (dugoročna podrška) i pratim Microsoft-ovu politiku podrške da planiram nadogradnje na vreme. Za legacy aplikacije planiram postepenu migraciju (npr. strangler pattern, .NET Upgrade Assistant) umesto „big bang“ prepisivanja.

Napomena za intervju: pre razgovora proverite na dotnet.microsoft.com koje su verzije trenutno u podršci.

P32. Objasnite async/await. Koje su tipične greške?

Šta se procenjuje: Razumevanje, ne samo sintakse.

async/await omogućava neblokirajuće I/O operacije: nit se oslobađa dok se čeka na bazu ili mrežu, pa server može da opsluži više zahteva sa istim brojem niti. Nije isto što i paralelizam (to je CPU-bound, Task.Run/Parallel).

Tipične greške:

- Blokiranje sa .Result ili .Wait() (rizik od deadlock-a i iscrpljivanja thread pool-a).
- async void van event handler-a (izuzeci se ne mogu uhvatiti).
- Zaboravljen await (zadatak se izvršava „u pozadini“ bez obrade grešaka).
- Nepropagiranje CancellationToken-a.
- Task.Run oko I/O operacija u ASP.NET Core-u bez potrebe.

```
// Loše: blokira nit
var user = _repo.GetUserAsync(id).Result;

// Dobro: async do kraja, sa tokenom za otkazivanje
public async Task<UserDto> GetAsync(int id, CancellationToken ct)
{
    var user = await _repo.GetUserAsync(id, ct);
    return user.ToDto();
}
```

P33. Kako radi Dependency Injection u ASP.NET Core-u? Razlika između Singleton, Scoped i Transient?

Šta se procenjuje: Osnove i poznavanje zamki.

Ugrađeni IoC kontejner registruje servise u Program.cs i ubrizgava ih kroz konstruktor.

- Transient: nova instanca pri svakom zahtevu za servis.
- Scoped: jedna instanca po HTTP zahtevu (tipično DbContext).

- Singleton: jedna instanca za celu aplikaciju (mora biti thread-safe).

Klasična greška je „captive dependency“: Singleton koji zavisi od Scoped servisa, pa Scoped instanca „živi“ predugo. Rešenje je IServiceScopeFactory ili promena životnog veka. Za HttpClient koristim IHttpClientFactory, ne ručno kreiranje instanci.

P34. Kako funkcioniše middleware pipeline u ASP.NET Core-u i zašto je redosled bitan?

Šta se procenjuje: Razumevanje request pipeline-a.

Svaki zahtev prolazi kroz lanac middleware komponenti, a svaka može da obradi zahtev, prosledi ga dalje i obradi odgovor na povratku. Redosled je bitan jer, na primer, UseAuthentication mora ići pre UseAuthorization, a middleware za obradu grešaka mora biti na početku da uhvati izuzetke iz kasnijih komponenti. Koristim ga za cross-cutting teme: logovanje, korelacioni ID, obrada grešaka (ProblemDetails), CORS, rate limiting.

P35. Entity Framework Core: kada ga koristite, a kada Dapper ili čist SQL? Kako izbegavate probleme sa performansama?

Šta se procenjuje: Praktično iskustvo sa podacima.

EF Core je odličan za produktivnost, migracije i tipičan CRUD, a Dapper ili čist SQL za složene izveštaje i kritične upite gde mi treba potpuna kontrola. Često kombinujem oba.

Najčešće zamke i rešenja:

- N+1 upiti: Include/projekcije (Select u DTO) umesto učitavanja celih entiteta.
- AsNoTracking za read-only upite.
- Paginacija i indeksi; analiza generisanog SQL-a (logging, execution plan).
- Dugo živeći DbContext (treba da bude Scoped, kratkog veka).
- Concurrency: rowversion/optimistic concurrency.
- Migracije u CI/CD sa pažnjom na zero-downtime promene šeme.

P36. Kako funkcioniše Garbage Collector u .NET-u i kako bi dijagnostikovali curenje memorije?

Šta se procenjuje: Dublje razumevanje runtime-a.

GC koristi generacije (0, 1, 2) jer većina objekata kratko živi; veliki objekti idu na Large Object Heap i skuplji su za sakupljanje. „Curenje“ u .NET-u obično znači objekte koji su i dalje referencirani: event handler-i koji nisu odjavljeni, statički keševi bez ograničenja, nepravilno oslobođeni IDisposable resursi.

Dijagnostika: dotnet-counters (praćenje GC i heap-a uživo), dotnet-gcdump/dotnet-dump i analiza heap-a, profileri (dotMemory, PerfView), a u produkciji metrike i alarmi na rast memorije. Za optimizaciju: ArrayPool, Span<T>/Memory<T>, izbegavanje nepotrebnih alokacija u vrućim putanjama.

P37. Minimal APIs ili kontroleri? Kako strukturišete veliki .NET projekat?

Šta se procenjuje: Arhitektonsko rasuđivanje.

Minimal APIs su lakši i brži za male servise i mikroservise, a kontroleri nude bogatiji ekosistem (filteri, model binding konvencije) i često su poznatiji većem timu; mogu se i kombinovati. Za strukturu volim vertical slice (funkcionalnosti grupisane zajedno) ili Clean Architecture sa jasnim granicama (Domain, Application, Infrastructure, API), u zavisnosti od složenosti domena. Najvažnije je da pravila zavisnosti budu jasna i da tim ima zajedničko razumevanje. Preterana arhitektura za jednostavan CRUD je takođe problem.

P38. Kako implementirate autentikaciju i autorizaciju u .NET API-ju?

Šta se procenjuje: Bezbednost i identity (stavka iz opisa).

Ne pravim sopstveni identity sistem ako ne moram: koristim OAuth 2.0 / OpenID Connect sa provajderom identiteta (npr. Microsoft Entra ID, Keycloak, Duende IdentityServer). API validira JWT access token (issuer, audience, potpis, rok trajanja).

- Autorizacija: role i, bolje, policy-based i claims-based pravila.
- Za servis-na-servis komunikaciju: client credentials flow ili managed identity, mTLS.
- Tajne u Key Vault-u, ne u kodu; kratkotrajni tokeni uz refresh.
- Zaštite: HTTPS svuda, rate limiting, validacija ulaza, OWASP Top 10 pregled u code review-u i u CI (SAST/dependency scan).

P39. Kako implementirate pozadinske poslove i asinhronu obradu u .NET-u?

Šta se procenjuje: Praktični obrasci za IoT/ingestion scenarije.

Za jednostavne periodične zadatke koristim BackgroundService/IHostedService, a za pouzdanu obradu red poruka (RabbitMQ, Azure Service Bus, Kafka) sa potrošačima koji su idempotentni, jer poruke mogu stići više puta. Za raspoređene poslove sa dashboard-om Hangfire ili Quartz. Ključno: retry sa exponential backoff-om, dead-letter red, graceful shutdown (poštovanje CancellationToken-a) i monitoring dužine reda. Za pouzdano objavljivanje događaja uz upis u bazu koristim Outbox pattern.

P40. Kako gradite otpornost (resilience) u komunikaciji između servisa?

Šta se procenjuje: Distribuirani sistemi.

Pretpostavljam da će spoljni pozivi otkazati. Koristim timeout-e, retry sa jitter-om (samo za idempotentne operacije), circuit breaker i bulkhead izolaciju, sve kroz Polly / Microsoft.Extensions.Http.Resilience na HttpClientFactory-u. Uz to health check endpoint-i (liveness/readiness), graceful degradation (npr. prikaz keširanih podataka) i idempotency ključevi za operacije koje menjaju stanje.

P41. Kako pristupate analizi performansi .NET aplikacije u produkciji?

Šta se procenjuje: Metodičnost, ne nagađanje.

Prvo merim, pa tek onda optimizujem. Počnem od metrika i tracing-a (OpenTelemetry, Application Insights, Datadog) da nađem usko grlo: baza, mreža, CPU ili alokacije. Zatim profilisanje (dotnet-trace, dotnet-counters, PerfView), analiza sporih upita i execution plan-ova, a mikrobenčmarkovi sa BenchmarkDotNet za vruće putanje. Tipični dobici: indeksi i bolji upiti, keširanje (IMemoryCache/Redis), smanjenje alokacija, batch-ovanje, pravilna upotreba async-a. Svaku optimizaciju potvrđujem merenjem pre i posle.

P42. Kako testirate .NET aplikacije?

Šta se procenjuje: Praktična veština.

xUnit (ili NUnit) za unit testove sa NSubstitute/Moq za zavisnosti; integracioni testovi sa WebApplicationFactory za API i Testcontainers za pravu bazu ili red poruka u Docker-u umesto lažnih. Za deterministične testove izdvajam vreme (TimeProvider) i slučajnost. U CI pokrećem testove na svakom PR-u, uz statičku analizu (analizatori, SonarQube) i proveru ranjivosti zavisnosti (dotnet list package --vulnerable).

P43. Koje SOLID principe najčešće viđate narušene i kako to rešavate sa timom?

Šta se procenjuje: Kvalitet koda, mentorstvo.

Najčešće: Single Responsibility (servisi sa hiljadu linija i „god“ klase) i Dependency Inversion (direktno kreiranje zavisnosti sa new). Rešavam kroz code review sa konkretnim primerima, male refaktorisanja (boy scout pravilo), parove na složenim delovima i zajedničke smernice. Ne forsiram principe dogmatski; pitanje je da li kod ostaje lak za menjanje i testiranje.

P44. Kako pratite i ocenjujete nove .NET funkcionalnosti i kada ih uvodite u tim?

Šta se procenjuje: Pragmatična inovacija.

Pratim release notes, Microsoft blog i zajednicu, a tim ohrabrujem da isprobava nove stvari kroz kratke spike-ove. Uvodim ih kada rešavaju stvarni problem (performanse, čitljivost, bezbednost), kada su stabilne i kada imamo plan obuke. Nadogradnje verzija radim redovno u malim koracima, da ne nagomilavamo dug i ostajemo u okviru podrške.

6. SaaS, cloud-native i IoT domen (praćenje elektro-mreže)

P45. Kako biste dizajnirali arhitekturu SaaS aplikacije koja prikuplja i vizualizuje podatke sa IoT uređaja?

Šta se procenjuje: System design razmišljanje (centralno pitanje za ovu poziciju).

Slojevi:

- Ingestion: uređaji šalju podatke preko MQTT/HTTPS/AMQP u IoT gateway (npr. Azure IoT Hub, AWS IoT Core ili Kafka/Event Hubs).
- Obrada: stream processing (validacija, obogaćivanje, detekcija anomalija) i pouzdan red poruka za razdvajanje proizvođača i potrošača.
- Skladištenje: time-series baza (TimescaleDB, InfluxDB, Azure Data Explorer) za sirove podatke uz downsampling/retention politike, relaciona baza za konfiguraciju i metapodatke, hladno skladište za arhivu.
- API sloj (.NET) sa keširanjem i paginacijom; web klijent (TypeScript) sa real-time osvežavanjem (SignalR/WebSocket).
- Cross-cutting: multi-tenancy, bezbednost, observability, CI/CD i infrastruktura kao kod.

Razgovor bih vodio kroz zahteve: broj uređaja, učestalost podataka, latencija, retencija, SLA, jer od njih zavise odluke.

P46. Kako rešavate multi-tenancy u SaaS aplikaciji?

Šta se procenjuje: SaaS iskustvo.

Tri osnovna modela: baza po tenantu (najjača izolacija, skuplje održavanje), šema po tenantu i deljena šema sa TenantId kolonom (najekonomičnije, rizik curenja podataka). Biram prema zahtevima izolacije, regulatornim zahtevima i skali; često hibrid (veliki klijenti izolovano). U .NET-u koristim globalne EF query filtere i tenant resolution middleware, uz automatske testove da nijedan upit ne vraća tuđe podatke. Posebna pažnja na „noisy neighbor“ problem (kvote, rate limiting) i na tenant-aware logovanje i metrike.

P47. Kada biste birali mikroservise, a kada modularni monolit?

Šta se procenjuje: Zrelost, izbegavanje hype-a.

Mikroservisi donose nezavisno skaliranje i deploy, ali i složenost (distribuirani sistemi, mreža, konzistentnost, operativni trošak). Za manji ili srednji tim obično počinjem sa modularnim monolitom sa jasnim granicama, i izdvajam servise tek kada postoji stvarna potreba: različite potrebe za skaliranjem (npr. ingestion pipeline), nezavisni timovi ili različiti ciklusi isporuke. Pogrešno je rano seći sistem bez jasnih granica domena (DDD, bounded contexts).

P48. Kako pristupate observability-ju i monitoringu (Prometheus, Datadog, New Relic)?

Šta se procenjuje: Poznavanje alata iz opisa posla.

Tri stuba: metrike, logovi i trace-ovi, povezani korelacionim ID-evima i standardizovani preko OpenTelemetry-ja da ne zavisimo od jednog vendora.

- Metrike: Prometheus + Grafana ili Datadog/New Relic; RED metrike za servise (rate, errors, duration) i USE za resurse.
- Alarmi vezani za SLO-ove i uticaj na korisnike, ne za svaku sitnicu (izbegavanje „alert fatigue“).
- Posebne metrike za IoT: broj aktivnih uređaja, kašnjenje podataka, stopa odbačenih poruka, dužina redova.
- Dashboard-i i runbook-ovi za dežurstvo.

P49. Koje bezbednosne i mrežne koncepte smatrate bitnim za ovakav sistem?

Šta se procenjuje: Bezbednost, identitet, mreža (stavka iz opisa).

Za kritičnu infrastrukturu bezbednost je primarna:

- Identitet uređaja: sertifikati po uređaju (X.509), mTLS, rotacija i opoziv ključeva, sigurno provisioning.
- Transport: TLS svuda; MQTT preko TLS-a; segmentacija mreže, VPC/VNet, privatni endpoint-i, firewall i WAF.
- Identitet korisnika: OAuth2/OIDC, SSO, MFA, RBAC i princip najmanjih privilegija; audit log.
- Podaci: enkripcija u mirovanju i u tranzitu, upravljanje tajnama (Key Vault), backup i plan oporavka.
- Proces: threat modeling, skeniranje zavisnosti i kontejnera, penetraciono testiranje, usklađenost sa standardima (npr. ISO 27001).

Kao lider obezbeđujem da bezbednost bude deo definicije „gotovo“, a ne naknadna provera.

P50. Kako biste obezbedili skalabilnost i pouzdanost kada broj uređaja naglo raste?

Šta se procenjuje: Skaliranje i kapacitet.

Horizontalno skaliranje stateless servisa (Kubernetes/App Service sa autoscaling-om), particionisanje podataka i redova (npr. po tenantu ili uređaju), backpressure i rate limiting da preopterećenje ne obori sistem, batch upisi u time-series bazu i keširanje za čitanja. Redovno radim load testove (k6, NBomber) sa realnim obrascima saobraćaja, definišem SLO-ove i kapacitetno planiranje, a za degradaciju scenarije planiram unapred (šta se dešava kada baza zaostaje).

P51. Kakvo je vaše iskustvo sa kontejnerima, CI/CD i infrastrukturom kao kodom?

Šta se procenjuje: Cloud-native iskustvo.

Docker za pakovanje, Kubernetes ili managed servisi za orkestraciju, pipeline-i (Azure DevOps/GitHub Actions/GitLab CI) koji build-uju, testiraju, skeniraju i deploy-uju automatski kroz okruženja, sa strategijama kao što su blue/green ili canary i feature flag-ovi. Infrastrukturu definišem kodom (Terraform/Bicep), tako da je ponovljiva i pregledljiva. Kao lider ne moram da sve radim sam, ali moram da razumem dovoljno da procenim rizike, postavim standarde i vodim razgovor sa DevOps/platform timom.

P52. Koja je vaša uloga u izboru tehnologija za frontend (TypeScript) i kako sarađujete sa frontend inženjerima?

Šta se procenjuje: Širina znanja, poverenje u tim.

Nisam frontend specijalista, ali razumem savremeni stack (TypeScript, React/Angular/Vue, state management, performanse velikih grafikona i vizualizacija podataka, pristupačnost). Odluke donosimo zajedno u timu, uz jasne kriterijume: zrelost, zajednica, iskustvo tima, performanse za vizualizaciju velikih skupova podataka. Brinem da API ugovori budu jasni (OpenAPI, generisani klijenti) da frontend i backend ne čekaju jedni druge.

P53. Koje skriptne jezike koristite za automatizaciju (Python, Bash, PowerShell) i zašto?

Šta se procenjuje: Poželjna veština iz opisa.

Bash i PowerShell za automatizaciju build-a, deploy-a i administrativnih zadataka, a Python za brze analize podataka, skripte za migracije i prototipove. Pravilo: ponavljajući ručni posao automatizujem, skripte držimo u repozitorijumu sa review-om i dokumentacijom, a ne na ličnim računarima.

7. Bihejvioralna pitanja (koristite STAR metodu)

P54. Ispričajte o situaciji kada ste doneli nepopularnu odluku.

Šta se procenjuje: Odgovornost, komunikacija, hrabrost.

Struktura STAR: Situacija, Zadatak, Akcija, Rezultat.

Primer odgovora: „Tim je želeo da uvede novi framework usred velikog izdanja. Procenio sam da je rizik prevelik jer smo imali fiksan rok. Objasnio sam razloge, priznao vrednost ideje, ponudio da je isprobamo kroz spike posle izdanja i dao vlasništvo za to inženjeru koji je predložio. Izdanje je isporučeno na vreme, a tehnologija je kasnije uvedena kontrolisano.“

Naglasite: transparentnost, saslušanje tima, preuzimanje odgovornosti.

P55. Opišite grešku koju ste napravili kao lider i šta ste naučili.

Šta se procenjuje: Samosvest i učenje.

Izaberite pravu grešku (ne „previše radim“). Na primer: „Predugo sam čekao da se pozabavim lošim performansama člana tima, nadajući se da će se samo popraviti. Tim je zbog toga trpeo. Naučio sam da rano, konkretno i direktno dajem povratnu informaciju, i sada imam navike redovnih 1:1 i jasnih očekivanja od početka.“

P56. Ispričajte kako ste unapredili proces ili timske performanse.

Šta se procenjuje: Kontinuirano unapređenje.

Pokažite problem, hipotezu, intervenciju i merljiv rezultat. Primer: „Deploy je trajao danima i bio ručni. Uveo sam CI/CD, automatizovane testove i feature flag-ove; učestalost isporuke je porasla sa jednom mesečno na više puta nedeljno, a broj incidenata posle deploy-a je pao.“ Ako nemate tačne brojke, procenite ih iskreno i recite da su okvirne.

P57. Kako ste vodili tim kroz veliku promenu (reorganizacija, migracija, odlazak ključne osobe)?

Šta se procenjuje: Upravljanje promenom.

Elementi dobrog odgovora: rano i iskreno komuniciranje, razumevanje zabrinutosti ljudi, jasan plan sa prekretnicama, uključivanje tima u detalje, brzi dobici da se gradi poverenje i redovno proveravanje raspoloženja. Za odlazak ključne osobe: transfer znanja, ponovna raspodela odgovornosti i prilika za rast drugima.

P58. Kada ste se ne složili sa nadređenim ili stakeholderom i kako ste to rešili?

Šta se procenjuje: Iskrenost uz profesionalnost (vrednost kompanije: otvorena i iskrena povratna informacija).

Primer: „Direktor je želeo ambiciozan rok. Doneo sam podatke o kapacitetu i rizicima i predložio tri opcije. Odabrali smo smanjen obim, a rok je ispoštovan. Naučio sam da neslaganje sa činjenicama i alternativama vredi više od običnog ‘ne’.“

P59. Kako ste pomogli članu tima da napreduje u karijeri?

Šta se procenjuje: Mentorstvo.

Opišite osobu, cilj, plan (mentorstvo, zadaci sa istezanjem, obuke, vidljivost), prepreke i ishod (npr. promocija ili prelazak na vođenje tima). Pokažite da vas uspeh drugih raduje i da merite sebe i kroz rast ljudi koje vodite.

8. Kratki odgovori na engleskom (poziciji treba tečan engleski)

P60. Tell me about yourself and your experience leading software teams.

Šta se procenjuje: Struktura: prošlost, sadašnjost, zašto ova uloga.

“I have X years of experience in software development, and the last Y years leading teams of Z engineers working on web-based SaaS products built with .NET and cloud technologies. My focus as a leader is building a high-trust, high-performing culture: clear expectations, strong engineering practices and continuous improvement. I enjoy growing people, and I am excited about this role because it combines technical leadership with real impact on the electrical grid.”

P61. Why Camlin, and why this role?

Šta se procenjuje: Pokažite da ste istražili kompaniju.

“Camlin is solving difficult real-world problems in power and rail, and the idea of building software that helps monitor critical grid infrastructure is meaningful to me. The role also matches how I like to work: managing people, supporting technical decisions and collaborating closely with product and project management. I share your values, especially open feedback and taking responsibility.”

P62. How do you handle a team member who disagrees with your decision?

Šta se procenjuje: Kratak, strukturiran odgovor.

“I welcome disagreement, as long as it is respectful and based on facts. I listen, ask questions and, if I still believe my decision is right, I explain my reasoning. I would then ask for commitment to the decision, even if there is no full agreement, and I stay open to revisiting it if new evidence appears.”

P63. How do you keep your team motivated and reduce attrition?

Šta se procenjuje: Kratak odgovor sa konkretnim merama.

“Regular one-to-ones, clear career paths, interesting problems, fair recognition and a safe environment to give honest feedback. I also watch for early warning signs of disengagement and act early, rather than waiting for a resignation letter.”

9. Pitanja koja vi postavljate kompaniji

P64. Pitanja za intervjuere (pokazuju zrelost i interes)

Šta se procenjuje: Postavite 3-5 najrelevantnijih.

- Kako izgleda tim koji bih vodio (veličina, senioritet, strukture) i koji su najveći izazovi u prvih 6-12 meseci?
- Kako se meri uspeh na ovoj poziciji nakon 3, 6 i 12 meseci?
- Koliko je tehničko vođstvo očekivano, a koliko administrativno i people management?
- Kako izgleda saradnja sa Product Ownerima, Project Managementom i drugim tehničkim liderima?
- Koji su trenutni tehnički prioriteti (migracije, tehnički dug, nove funkcionalnosti)?
- Kako izgleda proces zapošljavanja, planovi rasta tima i budžet za obuke?
- Kako se upravlja dežurstvima, incidentima i operativnom podrškom?
- Koji su najveći izazovi sa kojima se Software Engineering Director trenutno suočava?